Dart Basics

Table of Contents

What is Dart?....3
- How to run Dart code?....3
- Dart Features....4
  - Open Source....4
  - Platform Independent....4
  - Object-Oriented....4
  - Concurrency....4
  - Extensive Libraries....4
  - Flexible Compilation....5
  - Type Safe....5
  - Browser Support....5
- Dart installation and setup....5

What is Flutter?....6

Dart Basics....7
- Generating a Dart project with VSCode....7
- Comments....7
- Generating documentation....7
- Variables....9
  - Default value....9
  - Final and const....9
- Printing and String Interpolation....10
- User input....11
- Loops....12
  - While loop....12
  - for loop....12
  - do-while loop....12
  - For-in loop....12
  - Collection loop....12
- Enumerated types (enums)....12
- Data types....14
  - Automatic type....14
  - Dart Symbols....14
  - Dart Dynamic Type....14
  - Special types....14
  - Numbers....15
  - Strings....17
  - Boolean....17
  - Records....17
  - null and null safe coding....18

Object-Oriented Programming (OOP) in Dart....20
- Functions....20

Access modifiers....20
Public:....20
Private:....21
Classes....21
NOT null safe code....21
null safe code....22
Static variables....22
Getters and setters....23
Abstract methods....23
Abstract Classes....23
Objects....25
Getting an object's type....25
Inheritance....25
Polymorphism....26
Interfaces....26
Exception handling....28
Custom exceptions....29
Dart Collections....31
Lists (also known as arrays)....31
Removing duplicates....32
Sets....33
Maps....35
Queue:....36
LinkedList:....36
HashSet:....36
SplayTreeSet:....36
LinkedHashMap:....36
Generics....37
Operators....38
Spread operators....38
Control flow operators....39
Working with 'asynchronous' tasks....40
Using Future....42
Creating a Future:....42
Handling Future Results:....42
Handling Errors:....42
Using async and await:....42
What is Rest API....44
Key principles of RESTful APIs include:....44
Example of a REST API endpoint:....44
Dart "Future" with a simple Rest API access....46
main() code:....46
fetchData() function....46
Rest API to process a list of items....48
Reading a local file....49
Testing Dart Applications....50
References....53

What is Dart?

Dart is the open-source programming language originally developed by Google. It is meant for both server side as well as the user side. The Dart SDK comes with its compiler – the Dart VM and a utility dart2js which is meant for generating Javascript equivalent of a Dart Script so that it can be run on those sites also which don’t support Dart.

Dart is Object-oriented language and is quite similar to that of Java Programming. Dart is extensively use to create single-page websites and web-applications. Best example of dart application is Gmail.

- Everything in Dart is treated as an object including, numbers, Boolean, function, etc. like Python. All objects inherit from the Object class.
- Dart tools can report two types of problems while coding, warnings and errors. Warnings are the indication that your code may have some problem, but it doesn't interrupt the code's execution, whereas error can prevent the execution of code.
- Dart supports sound typing.
- Dart supports generic types, like List<int>(a list of integers) or List<dynamic> (a list of objects of any type).

How to run Dart code?

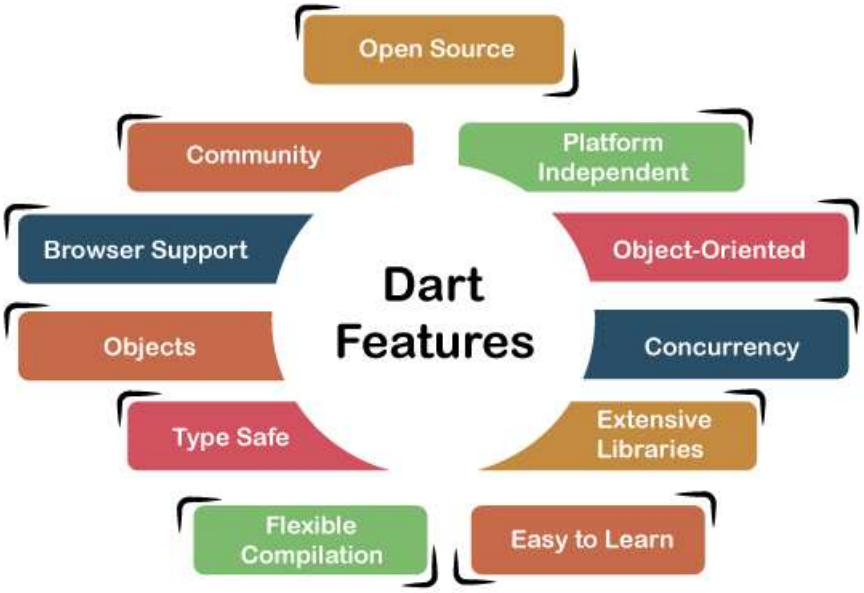
1. Online Compiler: The online compiler which support Dart is Dart Pad (https://dartpad.dev/)
2. IDE: The IDEs which support Dart are VSCode, Android Studio, WebStorm, Intellij, Eclipse, etc. Among them WebStorm from Jetbrains is available for Mac OS, Windows and Linux.
3. The dart program can also be compile through terminal by executing the code `dart file_name.dart`

Use the following command in the terminal to compile the Dart code into JavaScript code.

```
dart2js - - out = <output_file>.js  <dart_script>.dart
```

The above command will create a file that contains the JavaScript code corresponding to the Dart code.

Dart Features



Open Source

Dart is an open-source programming language, which means it is freely available. It is developed by Google, approved by the ECMA standard, and comes with a BSD license.

Platform Independent

Dart supports all primary operating systems such as Windows, Linux, Macintosh, etc. The Dart has its own Virtual Machine which known as Dart VM, that allows us to run the Dart code in every operating system.

Object-Oriented

Dart is an object-oriented programming language and supports all oops concepts such as classes, inheritance, interfaces and optional typing features. It also supports advance concepts like mixin, abstract, classes, reified generic, and robust type system.

Concurrency

Dart is an asynchronous programming language, which means it supports multithreading using Isolates. The isolates are the independent entities that are related to threads but don't share memory and establish the communication between the processes by the message passing. The message should be serialized to make effective communication. The serialization of the message is done by using a snapshot that is generated by the given object and then transmits to another isolate for deserialization .

Extensive Libraries

Dart consists of many useful inbuilt libraries including SDK (Software Development Kit), core, math, async, math, convert, html, IO, etc. It also provides the facility to organize the Dart code into libraries with proper namespacing. It can reuse by the import statement.

Flexible Compilation

Dart provides the flexibility to compile the code and fast as well. It supports two types of compilation processes, AOT (Ahead of Time) and JIT (Just-in-Time). The Dart code is transmitted in the other language that can run in the modern web-brewers.

Type Safe

The Dart is the type safe language, which means it uses both static type checking and runtime checks to confirm that a variable's value always matches the variable's static type, sometimes it known as the sound typing.

Browser Support

The Dart supports all modern web-browser. It comes with the dart2js compiler that converts the Dart code into optimized JavaScript code that is suitable for all type of web-browser.

Dart installation and setup

You can install Dart SDK from their official website (https://dart.dev/tools/sdk/archive) or download the installer from https://gekorm.com/dart-windows/

Follow the instructions here: https://www.geeksforgeeks.org/introduction-to-dart-programming-language/

What is Flutter?

Dart and Flutter are two related technologies primarily used for building cross-platform mobile, web, and desktop applications.

1. UI Toolkit:
   - Flutter is an open-source UI toolkit for building natively compiled applications for mobile, web, and desktop from a single codebase.

2. Developed by Google:
   - Flutter is developed and maintained by Google and is used by developers to create high-quality, performant applications.

3. Single Codebase:
   - With Flutter, you write your application code once and can deploy it to multiple platforms like iOS, Android, web, and desktop.

4. Widgets:
   - Flutter uses a reactive framework that revolves around widgets, which are the basic building blocks of the UI.
   - Widgets describe how the UI should look based on their current configuration and state.

5. Hot Reload:
   - Flutter includes a feature called Hot Reload, allowing developers to instantly see the effect of changes made to the code without restarting the app.

6. Integration with Dart:
   - Flutter is built using Dart, and Dart is the primary language for Flutter app development.

7. Growing Ecosystem:
   - Flutter has a growing ecosystem of packages and plugins that can be used to add functionality to your apps.

8. Community and Documentation:
   - Flutter has a strong and active community, and there is extensive documentation and resources available for developers.

In summary, Dart is a programming language, and Flutter is a UI toolkit built with Dart for creating cross-platform applications. Flutter has gained popularity for its expressive UI, single codebase approach, and support for a variety of platforms. Dart, on the other hand, is versatile and can be used beyond Flutter for various development purposes.

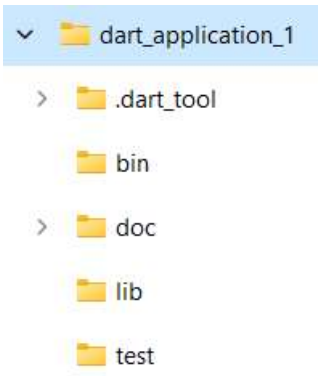
Dart Basics

Generating a Dart project with VSCode

Once dart is properly installed,

- Press Ctrl + Shift + P (Windows) or Cmd + Shift + P (Mac) to open the command palette.
- Type “Dart: New Project” and select it.
- Choose “Console Application” as the project type.
- Select a folder to create the project

Example project structure:



/bin contains the main Dart code

/doc contains API documents generated by you

/lib contains your custom libraries

/test contains test cases

Comments

```
//This is a single line comment

/**
 * This is a multi line comment
 */

/// this is a DocBlock comment

main() {
  print("Welcome to Dart");
}
```

Generating documentation

Type the following to generate documentation

```
dart doc
```

https://dart.dev/tools/dart-doc

https://dart.dev/effective-dart/documentation

Use the following code to generate Dart Doc:

```
/// This is a class documentation comment.
class MyClass {
  /// This is a variable documentation comment.
  int myVariable = 42;

  /// This is a function documentation comment.
  void myFunction() {
    // function implementation
  }
}
```

Variables

- The first character should not be a digit.
- Special characters are not allowed except underscore (_) or a dollar sign ($).
- Two successive underscores (__) are not allowed.
- The first character must be alphabet(uppercase or lowercase) or underscore.
- Identifiers must be unique and cannot contain whitespace.
- They are case sensitive. The variable name Joseph and joseph will be treated differently.

Below is the table of valid and invalid identifiers.

Valid Identifiers	Invalid Identifiers
firstname	__firstname
firstName	first name
var1	V5ar
$count	first-name
_firstname	1result
First_name	@var

Default value

Dart compiler provides default value (null) to the variable. Even the numeric type variables are initially assigned with the null value.

```
int count;
assert(count == null);
```

Final and const

It can be used in place of var or in addition to a type. A final variable can be set only one time where the variable is a compile-time constant.

```
final name = 'Ricky'; //final variable without type annotation.
final String msg = 'How are you?'; //with type annotation.
const a = 1000;
var f = const[];
```

Printing and String Interpolation

The print() function is used to print output on the console, and $expression is used for the string interpolation. Below is an example.

Example -

```
void main()
{
    var name = "Peter";
    var roll_no = 24;
    print("My name is ${name} My roll number is ${roll_no}");
}
```

Output:

```
My name is Peter My roll number is 24
```

User input

In Dart, you can get user input from the console using the stdin library, which is part of the `dart:io` library. Here's a simple example of how you can get user input. Run it from the console (NOT by VSCode “run” command, which gives errors).

```
import 'dart:io';

void main() {
  // Prompt the user for input
  stdout.write('Enter your name: ');

  // Read the user input from the console
  var userName = stdin.readLineSync();

  // Print a message using the user input
  print('Hello, $userName! How are you doing today?');
}
```

Keep in mind that this approach is suitable for console-based Dart applications. For Flutter applications or web-based Dart applications, you would typically use different methods to capture user input based on the platform and user interface components.

Loops

A looping statement in Dart is used to repeat a particular set of commands until certain conditions are not completed.

While loop

```
print('WHILE loop');
print('----------');
var max = 4;
int i = 1;
while (i <= max) {
  print('Hello Dart');
  i++;
}
```

for loop

```
print('\nFOR loop');
print('----------');
for (int i = 0; i < 5; i++) {
  print('I love Dart');
}
```

do-while loop

```
print('\nDO-WHILE loop');
print('---------------');
var count = 4;
int n = 1;
do {
  print('Good morning');
  n++;
} while (n <= count);
```

For-in loop

For…in loop in Dart takes an expression or object as an iterator. It is similar to that in Java and its execution flow is also the same as that in Java.

```
print('\nFOR-IN loop');
print('-------------');
var marks = [1, 2, 3, 4, 5];
for (int i in marks) {
 print(i);
}
```

Collection loop

The for-each loop iterates over all elements in some container/collectible and passes the elements to some specific function.

```
print('\nCollection loop');
print('-----------------');
var posts = [1, 2, 3, 4, 5];
posts.forEach((var one) => print(one));
```

Enumerated types (enums)

Enumerated types, often called enumerations or enums, are a special kind of class used to represent a fixed number of constant values.

```
// Define an enum named 'Color'
enum Color { red, green, blue }

void main() {
  // Using enum values
  Color favoriteColor = Color.blue;

  // Switch statement with enum
  switch (favoriteColor) {
    case Color.red:
      print('Red is your favorite color.');
      break;
    case Color.green:
      print('Green is your favorite color.');
      break;
    case Color.blue:
      print('Blue is your favorite color.');
      break;
    default:
      print('No favorite color selected.');
  }

  // Iterating over enum values
  print('\nAll enum values:');
  for (Color color in Color.values) {
    print(color);
  }
}
```

Data types

Each variable has its data-type. The Dart is a static type of language, which means that the variables cannot modify.

- Number
- Strings (UTF-16 unicode)
- Boolean
- Lists (also known as arrays)
- Maps
- Records
- Runes (often replaced by characters API)
- Symbols
- null/Null

Automatic type

The Dart compiler automatically knows the type of data based on the assigned to the variable because.

```
var <variable_name>  = <value>;
var <variable_name>;
var name = 'Andrew';
```

Dart Symbols

The Dart Symbols are the objects which are used to refer an operator or identifier that declare in a Dart program. It is commonly used in APIs that refers to identifiers by name because an identifier name can changes but not identifier symbols.

Dart Dynamic Type

Dart is an optionally typed language. If the variable type is not specified explicitly, then the variable type is dynamic. The dynamic keyword is used for type annotation explicitly.

```
//Unicode 16 test
void main() {
  var heart_symbol = '\u2665';
  var laugh_symbol = '\u{1f600}';
  print(heart_symbol);
  print(laugh_symbol);
}
```

Special types

Some other types also have special roles in the Dart language:

- Object: The superclass of all Dart classes except Null.
- Enum: The superclass of all enums.
- Future and Stream: Used in asynchrony support.
- Iterable: Used in for-in loops and in synchronous generator functions.

- Never: Indicates that an expression can never successfully finish evaluating. Most often used for functions that always throw an exception.
- dynamic: Indicates that you want to disable static checking. Usually you should use Object or Object? instead.
- void: Indicates that a value is never used. Often used as a return type.

The Object, Object?, Null, and Never classes have special roles in the class hierarchy.

Numbers

int

Integer values no larger than 64 bits, depending on the platform. On native platforms, values can be from -263 to 263 - 1. On the web, integer values are represented as JavaScript numbers (64-bit floating-point values with no fractional part) and can be from -253 to 253 - 1.

double

64-bit (double-precision) floating-point numbers, as specified by the IEEE 754 standard.

```
var x = 1;
var hex = 0xDEADBEEF;
var y = 1.1;
var exponents = 1.42e5;
double z = 1; // Equivalent to double z = 1.0.

num x = 1; // x can have both int and double values
x += 2.5;
```

Converting string to number and vice-versa

Converts the numeric string to the number.

```
void main() {
  String str1 = 'this is an example of a single-line string';
  String str2 = "this is an example of a double-quotes multiline line string";
  String str3 = """this is a multiline line
string using the triple-quotes""";

  var a = 10;
  var b = 20;

  print(str1);
  print(str2);
  print(str3);

  // We can add expression using the ${expression}.
  print("The sum is  = ${a + b}");

  // String -> int
  var one = int.parse('1');
  assert(one == 1);

// String -> double
  var onePointOne = double.parse('1.1');
  assert(onePointOne == 1.1);

// int -> String
```

```
  String oneAsString = 1.toString();
  assert(oneAsString == '1');

// double -> String
  String piAsString = 3.14159.toStringAsFixed(2);
  assert(piAsString == '3.14');
}
```

Output:

```
this is an example of a single-line string
this is an example of a double-quotes multiline line string
this is a multiline line
string using the triple-quotes
The sum is  = 30.5
```

Number Properties

Properties	Description
hashcode	It returns the hash code of the given number.
isFinite	If the given number is finite, then it returns true.
isInfinite	If the number infinite it returns true.
isNan	If the number is non-negative then it returns true.
isNegative	If the number is negative then it returns true.
sign	It returns -1, 0, or 1 depending upon the sign of the given number.
isEven	If the given number is an even then it returns true.
isOdd	If the given number is odd then it returns true.

Number Methods

Method	Description
abs()	It gives the absolute value of the given number.
ceil()	It gives the ceiling value of the given number.
floor()	It gives the floor value of the given number.
compareTo()	It compares the value with other number.
remainder()	It gives the truncated remainder after dividing the two numbers.
round()	It returns the round of the number.

toDouble()	It gives the double equivalent representation of the number.
toInt()	Returns the integer equivalent representation of the number.
toString()	Returns the String equivalent representation of the number
truncate()	Returns the integer after discarding fraction digits.

Strings

Dart String is a sequence of the character or UTF-16 code units. It is used to store the text value. The string can be created using single quotes or double-quotes. The multiline string can be created using the triple-quotes. Strings are immutable; it means you cannot modify it after creation.

```
var s1 = 'Single quotes work well for string literals.';
var s2 = "Double quotes work just as well.";
var s3 = 'It\'s easy to escape the string delimiter.';
var s4 = "It's even easier to use the other delimiter.";
```

Boolean

Only two objects have type bool: the boolean literals true and false, which are both compile-time constants.

```
// Check for an empty string.
var fullName = '';
assert(fullName.isEmpty);

// Check for zero.
var hitPoints = 0;
assert(hitPoints <= 0);

// Check for null.
var unicorn = null;
assert(unicorn == null);

// Check for NaN.
var iMeantToDoThis = 0 / 0;
assert(iMeantToDoThis.isNaN);
```

Records

Records expressions are comma-delimited lists of named or positional fields, enclosed in parentheses:

```
void main() {
  var record = ('first', a: 2, b: true, 'last');
  print("all: $record");

  print(record.$1); // Prints 'first'
  print(record.a); // Prints 2
  print(record.b); // Prints true
  print(record.$2); // Prints 'last'
}
```

Multiple returns

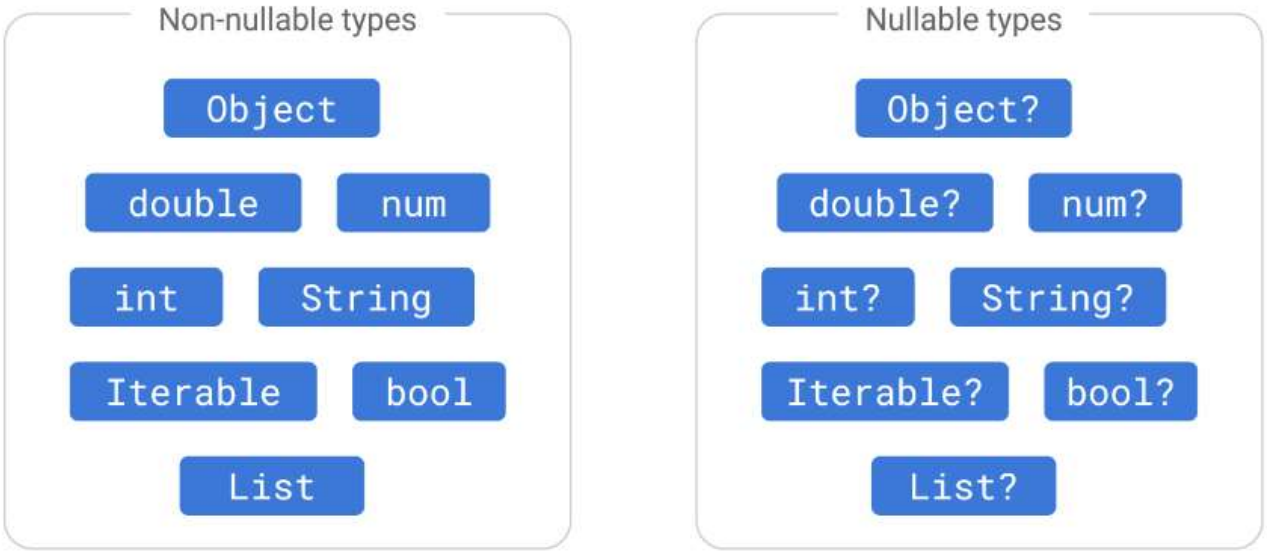
Records allow functions to return multiple values bundled together. To retrieve record values from a return, destructure the values into local variables using pattern matching.

```
// Returns multiple values in a record:
(String, int) userInfo(Map<String, dynamic> json) {
  return (json['name'] as String, json['age'] as int);
}

final json = <String, dynamic>{
  'name': 'Dash',
  'age': 10,
  'color': 'blue',
};

// Destructures using a record pattern:
var (name, age) = userInfo(json);

/* Equivalent to:
  var info = userInfo(json);
  var name = info.$1;
  var age  = info.$2;
*/
```

null and null safe coding

In Dart, null is a special value that represents the absence of a value. It is often used to indicate that a variable or expression does not have a meaningful result or does not refer to any object.

```
void main() {
  String nonNullableString = 'Hello'; // Non-nullable
  nonNullableString = null; //throws an error
  String? nullableString = null; // Nullable
}
```



What is null safety: https://dart.dev/null-safety/understanding-null-safety

Null safety in Dart - Introduction: https://youtu.be/iYhOU9AuaFs

Null Coalescing Operator (??):

The null coalescing operator (??) provides a concise way to use a default value when a variable is null.

```
String? username = getUsername();
String displayUsername = username ?? 'Guest';
```

Conditional Access (?.) and Null-aware Operators (??, ??=):

Dart provides null-aware operators to simplify working with nullable values, such as the conditional access operator (?.) and null-aware operators (??, ??=).

```
String? greeting = getGreeting();
int greetingLength = greeting?.length ?? 0;
```

Type Promotion with late and late final:

The late and late final keywords can be used to indicate that a variable will be assigned a non-null value before being used, allowing type promotion.

```
late String nonNullableString;
nonNullableString = 'Hello';
```

Object-Oriented Programming (OOP) in Dart

Functions

Dart is a true object-oriented language, so even functions are objects and have a type, Function. This means that functions can be assigned to variables or passed as arguments to other functions. You can also call an instance of a Dart class as if it were a function.

```
bool checkEven(n) {
  /// Returns true
  /// if n is even
  if (n % 2 == 0)
    return true;

  /// Returns false if n is odd
  else
    return false;
}

int main() {
  int n = 44;
  print(checkEven(n));
  return 0;
}
```

For functions that contain just one expression, you can use a shorthand syntax:

```
//using arrow syntax
bool checkEven(n) => n % 2 == 0;

int main() {
  int n = 43;
  print(checkEven(n));
  return 0;
}
```

The => expr syntax is a shorthand for { return expr; }. The => notation is sometimes referred to as arrow syntax.

Access modifiers

Dart has a concept of public and private members, and the visibility of a member is determined by the presence or absence of an underscore (_) before its name. In Dart, there is no explicit “protected” access modifier like other programming languages (eg: Java, C++, C#).

Public:

Members without an underscore are considered public and can be accessed from anywhere.

```
class MyClass {
  int publicVariable = 42;

  void publicMethod() {
    print('This is a public method.');
  }
}
```

Private:

Members with an underscore at the beginning of their names are considered private to their library. Dart doesn't have true "private" in the sense of class-level scoping; instead, it uses library-level privacy.

```
class MyClass {
  int _privateVariable = 42;

  void _privateMethod() {
    print('This is a private method.');
  }
}
```

A private member is accessible only within the same Dart library. Dart considers all code in a single file to be part of the same library, so members marked with an underscore can be accessed within the same file but not from outside.

Keep in mind that Dart's privacy is based on the concept of libraries rather than classes. If you want to hide implementation details, you can use separate files or directories to organize your code into different libraries.

Classes

Dart is an object-oriented language with classes and mixin-based inheritance. Every object is an instance of a class, and all classes except Null descend from Object.

NOT null safe code

```
class Person {
  // Fields or properties
  String name;
  int age;
  String _privateField; // Private field, denoted by an underscore

  // Constructor
  Person(this.name, this.age);

  // Named Constructor
  Person.withPrivateField(this._privateField);

  // Getter for private field
  String get privateField => _privateField;

  // Setter for private field
  set privateField(String value) {
    _privateField = value;
  }

  // Method
  void sayHello() {
    print('Hello, my name is $name, and I am $age years old.');
  }
}

void main() {
  // Creating an instance of the Person class
  Person person1 = Person('Alice', 25);
```

```
  // Accessing fields and calling methods
  print('${person1.name} is ${person1.age} years old.');
  person1.sayHello();

  // Creating an instance using the named constructor
  Person person2 = Person.withPrivateField('PrivateData');
  print('Private field value: ${person2.privateField}');
  person2.privateField = 'UpdatedPrivateData';
  print('Updated private field value: ${person2.privateField}');
}
```

null safe code

```
class Person {
  // Fields or properties. These can be nulled (see the "?")
  String? name;
  int? age;
  String? _privateField; // Private field, denoted by an underscore

  // Constructor
  Person(this.name, this.age);

  // Named Constructor
  Person.withPrivateField(this._privateField);

  // Getter for private field
  String get privateField => _privateField ?? "";

  // Setter for private field
  set privateField(String value) {
    _privateField = value;
  }

  // Method
  void sayHello() {
    print('Hello, my name is $name, and I am $age years old.');
  }
}

void main() {
  // Creating an instance of the Person class
  Person person1 = Person('Alice', 25);

  // Accessing fields and calling methods
  print('${person1.name} is ${person1.age} years old.');
  person1.sayHello();

  // Creating an instance using the named constructor
  Person person2 = Person.withPrivateField('PrivateData');
  print('Private field value: ${person2.privateField}');
  person2.privateField = 'UpdatedPrivateData';
  print('Updated private field value: ${person2.privateField}');
}
```

Static variables

Static variables (class variables) are useful for class-wide state and constants:

```
class Queue {
  static const initialCapacity = 16;
  // ···
}
```

```
void main() {
  assert(Queue.initialCapacity == 16);
}
```

Static variables aren’t initialized until they’re used.

Getters and setters

Getters and setters are special methods that provide read and write access to an object’s properties. Recall that each instance variable has an implicit getter, plus a setter if appropriate. You can create additional properties by implementing getters and setters, using the get and set keywords:

```
class Rectangle {
  double left, top, width, height;

  Rectangle(this.left, this.top, this.width, this.height);

  // Define two calculated properties: right and bottom.
  double get right => left + width;
  set right(double value) => left = value - width;
  double get bottom => top + height;
  set bottom(double value) => top = value - height;
}

void main() {
  var rect = Rectangle(3, 4, 20, 15);
  assert(rect.left == 3);
  rect.right = 12;
  assert(rect.left == -8);
}
```

With getters and setters, you can start with instance variables, later wrapping them with methods, all without changing client code.

Abstract methods

Instance, getter, and setter methods can be abstract, defining an interface but leaving its implementation up to other classes. Abstract methods can only exist in abstract classes or mixins.

To make a method abstract, use a semicolon (;) instead of a method body:

```
abstract class Doer {
  // Define instance variables and methods...

  void doSomething(); // Define an abstract method.
}

class EffectiveDoer extends Doer {
  void doSomething() {
    // Provide an implementation, so the method is not abstract here...
  }
}
```

Abstract Classes

An abstract class is a class that cannot be instantiated directly and is typically used as a base class for other classes. Abstract classes can have abstract methods (methods without a body) that must be implemented by concrete (non-abstract) subclasses.

```
// Define an abstract class named 'Shape'
abstract class Shape {
  // Abstract method to calculate area
  double calculateArea();

  // Regular method
  void printDescription() {
    print('This is a shape.');
  }
}

// Concrete subclass 'Circle' that extends 'Shape'
class Circle extends Shape {
  double radius;

  // Constructor for Circle
  Circle(this.radius);

  // Implementation of the abstract method 'calculateArea' for Circle
  @override
  double calculateArea() {
    return 3.14 * radius * radius;
  }
}

// Concrete subclass 'Rectangle' that extends 'Shape'
class Rectangle extends Shape {
  double length;
  double width;

  // Constructor for Rectangle
  Rectangle(this.length, this.width);

  // Implementation of the abstract method 'calculateArea' for Rectangle
  @override
  double calculateArea() {
    return length * width;
  }

  // Overriding the 'printDescription' method
  @override
  void printDescription() {
    print('This is a rectangle with length $length and width $width.');
  }
}

void main() {
  // Create instances of concrete classes
  Circle circle = Circle(5.0);
  Rectangle rectangle = Rectangle(4.0, 6.0);

  // Call methods on instances
  circle.printDescription();
  print('Area of the circle: ${circle.calculateArea()}');

  print('');

  rectangle.printDescription();
  print('Area of the rectangle: ${rectangle.calculateArea()}');
}
```

Objects

Getting an object's type

To get an object's type at runtime, you can use the Object property runtimeType, which returns a Type object.

```
print('The type of a is ${a.runtimeType}');
```

Inheritance

Class inheritance allows one class to inherit properties and methods from another class.

```
// Base class
class Animal {
  String name;
  int age;

  Animal(this.name, this.age);

  void makeSound() {
    print('Animal makes a sound');
  }
}

// Subclass (derived class) inheriting from Animal
class Dog extends Animal {
  String breed;

  // Constructor for the Dog class
  Dog(String name, int age, this.breed) : super(name, age);

  // Overriding the makeSound method
  @override
  void makeSound() {
    print('Dog barks: Woof! Woof!');
  }

  // Additional method specific to the Dog class
  void wagTail() {
    print('Dog wags its tail');
  }
}

void main() {
  // Creating an object of the Dog class
  Dog myDog = Dog('Buddy', 3, 'Labrador');

  // Accessing properties and calling methods from the base class
  print('${myDog.name} is ${myDog.age} years old.');
  myDog.makeSound();

  // Accessing properties and calling methods from the derived class
  print('Breed: ${myDog.breed}');
  myDog.wagTail();
}
```

Polymorphism

Polymorphism allows objects of different classes to be treated as objects of a common base class. This is achieved through method overriding. Here's an example demonstrating polymorphism using a base class Shape and its subclasses Circle and Square.

```
// Base class
class Shape {
  void draw() {
    print('Drawing a shape.');
  }
}

// Subclass 1
class Circle extends Shape {
  @override
  void draw() {
    print('Drawing a circle.');
  }
}

// Subclass 2
class Square extends Shape {
  @override
  void draw() {
    print('Drawing a square.');
  }
}

// Function using polymorphism
void drawShape(Shape shape) {
  shape.draw();
}

void main() {
  // Creating objects of different subclasses
  Circle myCircle = Circle();
  Square mySquare = Square();

  // Using polymorphism in function calls
  drawShape(myCircle); // Output: Drawing a circle.
  drawShape(mySquare); // Output: Drawing a square.
}
```

Interfaces

Interfaces are defined by creating abstract classes. A class can implement one or more interfaces by providing implementations for the methods declared in those interfaces.

```
// Define an interface
abstract class Animal {
  void makeSound();
}

// Implement the interface in a class
class Dog implements Animal {
  String name;

  // Constructor
  Dog(this.name);

  // Implement the method from the interface
```

```
  @override
  void makeSound() {
    print('$name says Woof!');
  }
}

// Implement another interface
abstract class Flyable {
  void fly();
}

// Implement multiple interfaces in a class
class Bird implements Animal, Flyable {
  String name;

  // Constructor
  Bird(this.name);

  // Implement the methods from the interfaces
  @override
  void makeSound() {
    print('$name sings beautifully!');
  }

  @override
  void fly() {
    print('$name is flying.');
  }
}

void main() {
  // Create objects of classes that implement interfaces
  Dog myDog = Dog('Buddy');
  Bird myBird = Bird('Robin');

  // Call methods defined in the interfaces
  myDog.makeSound();
  myBird.makeSound();
  myBird.fly();
}
```

Exception handling

Exception handling in Dart is the process of dealing with errors that may occur during the execution of a program. Dart provides a mechanism to throw and catch exceptions, allowing developers to gracefully handle errors and prevent them from causing the program to crash.

1. Throwing Exceptions:

- You can throw an exception using the `throw` keyword followed by an instance of an exception class.
- Dart provides several built-in exception classes, such as `Exception` and `Error`, which can be extended to create custom exceptions.

```
void exampleFunction() {
  throw Exception('This is an example exception.');
}
```

2. Catching Exceptions:

- To catch exceptions, use a `try`, `catch`, and optionally, `finally` block.
  - The `try` block contains the code where an exception might occur.
  - The `catch` block is executed if an exception occurs in the `try` block.
  - The `finally` block, if present, is executed regardless of whether an exception occurred or not.

```
try {
  // Code that might throw an exception
  exampleFunction();
} catch (e) {
  // Code to handle the exception
  print('Caught an exception: $e');
} finally {
  // Code that will always be executed
  print('Finally block executed.');
}
```

3. On Clause:

- Dart also supports the `on` clause to catch exceptions of a specific type.
- This is useful when you want to handle different exceptions differently.

```
try {
  // Code that might throw an exception
  exampleFunction();
} on Exception catch (e) {
  // Code to handle the specific exception type
  print('Caught an Exception: $e');
} catch (e) {
  // Code to handle any other type of exception
  print('Caught an unknown exception: $e');
}
```

4. Re-throwing Exceptions:

- You can re-throw an exception after catching it using the `rethrow` keyword.
- This is useful when you want to catch an exception, perform some logging or cleanup, and then propagate the exception.

```
try {
  // Code that might throw an exception
  exampleFunction();
} catch (e) {
  // Code to handle the exception
  print('Caught an exception: $e');
  // Re-throwing the exception
  rethrow;
}
```

Exception handling in Dart allows developers to gracefully handle errors, log relevant information, and maintain the stability of their applications. It's an essential aspect of writing robust and reliable Dart code.

Custom exceptions

You can create a custom exception class by extending the built-in `Exception` class or any of its subclasses. Custom exception classes are useful when you want to represent specific error conditions in your code. Here's an example:

```
// Custom exception class extending the built-in Exception class
class CustomException extends Exception {
  final String message;

  CustomException(this.message);

  @override
  String toString() {
    return 'CustomException: $message';
  }
}

void main() {
  // Example of throwing and catching a custom exception
  try {
    throw CustomException('This is a custom exception example.');
  } catch (e) {
    print('Caught an exception: $e');
  }
}
```

In this example:

- The `CustomException` class extends the built-in `Exception` class.
- It has a constructor that takes a `message` parameter to provide additional information about the exception.
- The `toString` method is overridden to customize the string representation of the exception.

In the `main` function:

- A custom exception is thrown using the `throw` keyword with an instance of `CustomException`.

- The exception is caught in a `try-catch` block, and the caught exception is printed.

This is a basic example, and in a real-world scenario, you might include additional fields or methods in your custom exception class to provide more context or behavior related to the specific error condition it represents. Custom exceptions can help improve the clarity of your code and make error handling more expressive.

Dart Collections

Dart provides a variety of collection types for working with groups of objects or values. Dart also provides several utility functions and methods for working with collections, such as forEach, map, where, fold, and more. These functions make it easy to perform common operations on collections in a concise and expressive manner.

Keep in mind that Dart's collection types are generic, allowing you to specify the type of elements they can contain, which adds a layer of type safety to your code.

Lists (also known as arrays)

A very commonly used collection in programming is an array. Dart represents arrays in the form of List objects. A List is simply an ordered group of objects. Lists can contain duplicate values.

```
void main() {
  // Creating a List of integers
  List<int> numbers = [1, 2, 3, 4, 5];

  // Printing the entire list
  print("Numbers: $numbers");

  // Accessing elements in the list
  print("First element: ${numbers[0]}");
  print("Second element: ${numbers[1]}");

  // Modifying elements in the list
  numbers[2] = 10;
  print("Modified list: $numbers");

  // Adding elements to the end of the list
  numbers.add(6);
  numbers.add(7);
  print("List after adding elements: $numbers");

  // Removing an element from the list
  numbers.remove(4);
  print("List after removing element 4: $numbers");

  //list after sorting
  numbers.sort();
  print("List after sorting: $numbers");

  // Checking if the list contains a specific element
  bool containsThree = numbers.contains(3);
  print("Does the list contain 3? $containsThree");

  //no of elements
  int size = numbers.length;
  print("No of elements: $size");

  // Iterating over the list using a for-in loop
  print("Iterating over the list with for-in:");
  for (int number in numbers) {
    print(number);
  }

  // Iterating over the list using a collection loop
  print("Iterating over the list with forEach:");
```

```
  numbers.forEach((var one) => (print(one)));
}
```

Removing duplicates

```
List<T> removeDuplicates<T>(List<T> inputList) {
  // Use a Set to keep track of unique elements
  Set<T> uniqueElements = {};

  // Create a new list with unique elements
  List<T> result = inputList.where((element) {
    // Add the element to the set if it hasn't been encountered before
    return uniqueElements.add(element);
  }).toList();

  return result;
}

void main() {
  // Example usage:
  List<int> numbers = [1, 2, 3, 4, 2, 5, 6, 3, 7, 8, 1];

  // Remove duplicates from the list
  List<int> uniqueNumbers = removeDuplicates(numbers);

  // Print the original and modified lists
  print('Original List: $numbers');
  print('List with Duplicates Removed: $uniqueNumbers');
}
```

Sets

A Set is an unordered collection of unique items.

Example 1: Creating a set of numbers

```
Set<int> numbers = {1, 2, 3, 4, 5};
```

This code creates a set of integers named numbers and adds the numbers 1, 2, 3, 4, and 5 to the set.

Example 2: Checking if an element is in a set

```
Set<String> fruits = {'apple', 'banana', 'orange'};
bool isAppleInSet = fruits.contains('apple');
print(isAppleInSet); // Output: true
print(fruits.length); //shows how many elements are there: 3
```

This code checks if the element 'apple' is in the set fruits. The output of the code will be true.

Example 3: Adding an element to a set

```
Set<String> animals = {'dog', 'cat', 'rabbit'};
animals.add('hamster');
print(animals); // Output: {dog, cat, rabbit, hamster}
animals.addAll({'bird', 'monkey', 'zebra'});
print(animals); // Output: {dog, cat, rabbit, hamster, bird, monkey, zebra}
```

This code adds the element 'hamster' to the set animals. The output of the code will be a set containing the elements 'dog', 'cat', 'rabbit', and 'hamster'.

Example 4: Removing an element from a set

```
Set<String> colors = {'red', 'green', 'blue', 'yellow'};
colors.remove('yellow');
print(colors); // Output: {red, green, blue}
```

This code removes the element 'yellow' from the set colors. The output of the code will be a set containing the elements 'red', 'green', and 'blue'.

Example 5: Iterating over a set

```
Set<String> countries = {'India', 'China', 'USA', 'Brazil'};
for (String country in countries) {
  print(country);
}
```

This code iterates over the set countries and prints each element of the set to the console. The output of the code will be:

```
India
China
USA
Brazil
```

In Dart, you can't directly sort a Set because a Set is an unordered collection. However, you can convert a Set to a List, sort the list, and then convert it back to a Set.

```
void main() {
```

```
  // Creating a Set of integers
  Set<int> numberSet = {5, 2, 8, 3, 1};

  // Convert the Set to a List
  List<int> sortedList = numberSet.toList();

  // Sort the List
  sortedList.sort();

  // Convert the sorted List back to a Set
  Set<int> sortedNumberSet = Set<int>.from(sortedList);

  print("Original Set: $numberSet");
  print("Sorted Set: $sortedNumberSet");
}
```

Maps

A Map is a collection of key-value pairs. Each key must be unique.

```
void main() {
  // Creating a Map
  Map<String, int> studentGrades = {
    'Alice': 90,
    'Bob': 85,
    'Charlie': 92,
    'David': 88,
  };

  // Accessing values in the Map
  print('Alice\'s grade: ${studentGrades['Alice']}');

  // Modifying a value in the Map
  studentGrades['Bob'] = 89;

  // Adding a new entry to the Map
  studentGrades['Eva'] = 95;

  // Removing an entry from the Map
  studentGrades.remove('David');

  // Iterating over the Map
  print('\nStudent Grades:');
  studentGrades.forEach((key, value) {
    print('$key: $value');
  });
}
```

There are 2 ways to create maps, the constructor method is preferred.

```
//created using map literals:
var gifts = {
  // Key:    Value
  'first': 'partridge',
  'second': 'turtledoves',
  'fifth': 'golden rings'
};

var nobleGases = {
  2: 'helium',
  10: 'neon',
  18: 'argon',
};

//using a Map constructor:
var gifts = Map<String, String>();
gifts['first'] = 'partridge';
gifts['second'] = 'turtledoves';
gifts['fifth'] = 'golden rings';

var nobleGases = Map<int, String>();
nobleGases[2] = 'helium';
nobleGases[10] = 'neon';
nobleGases[18] = 'argon';
```

Queue:

A collection representing a queue (first-in, first-out).

```
Queue<int> queue = Queue<int>()..addAll([1, 2, 3, 4]);
```

LinkedList:

A doubly-linked list that allows efficient insertions and removals.

```
LinkedList<int> linkedList = LinkedList<int>()..addAll([1, 2, 3, 4]);
```

HashSet:

An unordered set that uses hash values to organize elements.

```
HashSet<String> hashSet = HashSet<String>.from(['apple', 'banana', 'orange']);
```

SplayTreeSet:

A self-balancing binary search tree set.

```
SplayTreeSet<int> splayTreeSet = SplayTreeSet<int>.from([5, 2, 8, 1]);
```

LinkedHashMap:

A hash map with predictable ordering based on insertion order.

```
LinkedHashMap<String, int> linkedHashMap = LinkedHashMap<String, int>()
  ..addAll({'one': 1, 'two': 2, 'three': 3});
```

Generics

Dart supports generics, allowing you to write code that can work with different data types while maintaining type safety.

```
// A generic class that can hold any type of data
class Box<T> {
  T value;

  Box(this.value);

  void display() {
    print('Box contains: $value');
  }
}

void main() {
  // Creating a Box for integers
  var integerBox = Box<int>(42);
  integerBox.display();

  // Creating a Box for strings
  var stringBox = Box<String>('Dart is awesome!');
  stringBox.display();

  // Creating a Box for doubles
  var doubleBox = Box<double>(3.14);
  doubleBox.display();
}
```

Operators

Spread operators

Dart supports the spread operator (...) and the null-aware spread operator (...?) in list, map, and set literals. Spread operators provide a concise way to insert multiple values into a collection.

For example, you can use the spread operator (...) to insert all the values of a list into another list:

```
var list = [1, 2, 3];
var list2 = [0, ...list];
assert(list2.length == 4);
```

If the expression to the right of the spread operator might be null, you can avoid exceptions by using a null-aware spread operator (...?):

```
var list2 = [0, ...?list];
assert(list2.length == 1);
```

Control flow operators

Dart offers collection if and collection for for use in list, map, and set literals. You can use these operators to build collections using conditionals (if) and repetition (for).

```
void main() {
  //Here’s an example of using collection if to create a list with three or four items in it:
  bool promoActive = true;
  var nav1 = ['Home', 'Furniture', 'Plants', if (promoActive) 'Outlet'];
  print("nav1: $nav1");

  //Dart also supports if-case inside collection literals:
  String role = 'Manager';
  var nav2 = [
    'Home',
    'Furniture',
    'Plants',
    if (role case 'Manager') 'Inventory'
  ];
  print("nav2: $nav2");

  //Here’s an example of using collection for to manipulate the items of a list before adding
them to another list:
  var listOfInts = [1, 2, 3];
  var listOfStrings = ['#0', for (var i in listOfInts) '#$i'];
  print(listOfStrings);
}
```

Working with ‘asynchronous’ tasks

Dart's async and await keywords are used to work with asynchronous code, making it more readable and easier to reason about. Let's break down Dart's asynchronous programming model in a way that's easy to understand.

What is Asynchronous Programming?

In programming, tasks are often performed one after the other, in a sequence. This is called synchronous programming. Asynchronous programming allows us to start a task, then move on to do other things while waiting for the first task to finish.

Futures: Representing Future Values

In Dart, we use something called a Future to represent a value (or an error) that will be available at some time in the future. It's like telling Dart, "Hey, I'm going to do something that might take a while. Here's a 'future' value that you can use when I'm done."

Asynchronous Functions with async and await:

When we have a function that performs an asynchronous operation, we mark that function with the async keyword. We use the await keyword to tell Dart to wait for the completion of an asynchronous operation before moving on.

Example: Waiting for a Delayed Task:

```
//main code: bin/future_test.dart
import 'package:future_test/fetch_data.dart' as future_test;

void main(List<String> arguments) {
  print('Testing Dart async/await system.');

  future_test.fetchData().then(
        (value) =>
            print(value), // Called when the Future completes successfully
        onError: (error) =>
            print("Error: $error"), // Called when there's an error
      );
  print("This line executes immediately");
}

//library function: /lib/fetch_data.dart
Future<String> fetchData() {
  // Simulating an asynchronous operation
  return Future.delayed(
      Duration(seconds: 5), () => 'Data fetched after 5 seconds successfully');
}
```

Here, Future.delayed simulates a task that takes 1 second to complete.

The await keyword makes sure that the program waits for this task to finish before moving on to the next line.

Event Loop: The Brain Behind Asynchrony:

Dart has something called an "event loop" that keeps track of all the tasks that are running.

It checks if a task is done, and if so, it executes the code associated with that task.

Why Use Asynchronous Programming in Dart?

Efficiency: While waiting for one task to finish, Dart can start working on other tasks, making our programs more efficient.

Responsive UI: In apps, we can keep the user interface responsive even when performing time-consuming tasks.

Remember: It's Like Cooking:

Imagine you're cooking. You put something on the stove to simmer. Instead of staring at the pot until it's done, you can start chopping vegetables or setting the table.

Similarly, in Dart, we can start tasks, then work on other things while waiting for the initial tasks to complete.

Dart's asynchronous model is like having a personal assistant that can manage multiple tasks for you, allowing your program to do more things at the same time without getting stuck.

Using Future

A Future is a placeholder for a value that will be available at some point. It represents the eventual completion or failure of an asynchronous operation. It's a fundamental concept for handling asynchronous operations in Dart, such as I/O operations, network requests, or timer callbacks.

Creating a Future:

You can create a Future using the Future constructor or by using one of the utility functions provided by the dart:async library, such as Future.value or Future.delayed.

```
Future<String> fetchData() {
  // Simulating an asynchronous operation
  return Future.delayed(Duration(seconds: 2), () => 'Data fetched successfully');
}
```

In this example, fetchData is a function that returns a Future<String>. The Future.delayed function simulates an asynchronous operation that completes after a delay of 2 seconds.

Handling Future Results:

You can use the then method to handle the result of a Future when it's available.

```
fetchData().then((result) {
  print(result); // Output: Data fetched successfully
});
```

The then method takes a callback that will be invoked when the Future completes with a result.

Handling Errors:

You can use the onError: method to handle errors that occur during the execution of a Future.

```
void main() {
  fetchData().then(
    (value) => print(value), // Called when the Future completes successfully
    onError: (error) => print("Error: $error"), // Called when there's an error
  );
  print("Fetching data..."); // This line executes immediately
}
```

In this example, "Fetching data..." is printed immediately, and when the fetchData Future completes (after 2 seconds), either the success callback or the error callback is invoked.

Using async and await:

Dart also provides the async and await keywords, which make it easier to work with asynchronous code. The async keyword is used to mark a function as asynchronous, and the await keyword is used to wait for a Future to complete.

```
void main() async {
  try {
    var result = await fetchData();
    print(result);
  } catch (error) {
    print("Error: $error");
  }
  print("Fetching data..."); // This line executes after fetchData completes
```

```
}
```

In this example, the await keyword is used to pause execution until the fetchData Future completes, making the code appear more synchronous.

Understanding and effectively using Future is crucial in Dart asynchronous programming, especially in scenarios where you need to handle operations such as fetching data from a server, reading a file, or performing other time-consuming tasks without blocking the main thread.

What is Rest API

REST API, which stands for Representational State Transfer Application Programming Interface, is a set of conventions or architectural principles for building and interacting with web services. REST is not a technology but rather an architectural style that uses a stateless, client-server communication model. It is commonly used in web development to enable communication between different systems over the internet.

Key principles of RESTful APIs include:

- Statelessness: Each request from a client to a server must contain all the information needed to understand and process the request. The server should not store any information about the client's state between requests. This enhances scalability and simplifies the design.
- Client-Server Architecture: The client and server are separate entities that communicate over a network. The client is responsible for the user interface and user experience, while the server is responsible for processing requests and managing resources.
- Uniform Interface: RESTful APIs have a uniform and standardized way of interacting with resources. This includes a consistent use of HTTP methods (GET, POST, PUT, DELETE) for different operations and a resource-based approach where each resource is identified by a unique URI (Uniform Resource Identifier).
- Representation: Resources are represented in a format that can be easily processed by the client, such as JSON or XML. The representation of a resource contains both data and metadata.
- Stateless Communication: Each request from a client to a server must contain all the information needed to understand and process the request. The server should not store any information about the client's state between requests.

A typical REST API allows clients to perform CRUD (Create, Read, Update, Delete) operations on resources, where resources can be any identifiable entity, such as users, products, or documents. The API is accessed through standard HTTP methods, and the communication is usually done using JSON or XML for data interchange.

Example of a REST API endpoint:

`GET /api/users`	Get a list of all users
`POST /api/users`	Store one new user record
`GET /api/users/{id}`	Get the user record given by the id, eg: `GET /api/users/1`
`PUT /api/users/{id}`	Update the user record given by the id
`DELETE /api/users/{id}`	Delete the user record given by the id

In this example, /api/users represents a collection of users, and clients can perform various operations on this resource using different HTTP methods. The {id} part in the URI allows the client to specify a particular user by its identifier.

RESTful APIs have become the predominant choice for building web services due to their simplicity, scalability, and ease of use. They are widely used in web and mobile application development for enabling communication between clients and servers.

Use the “Thunder Client” plugin with VSCode and experiment the rest API endpoints available at: https://jsonplaceholder.typicode.com/guide/

Dart “Future” with a simple Rest API access

More info here: https://dart.dev/tutorials/server/fetch-data

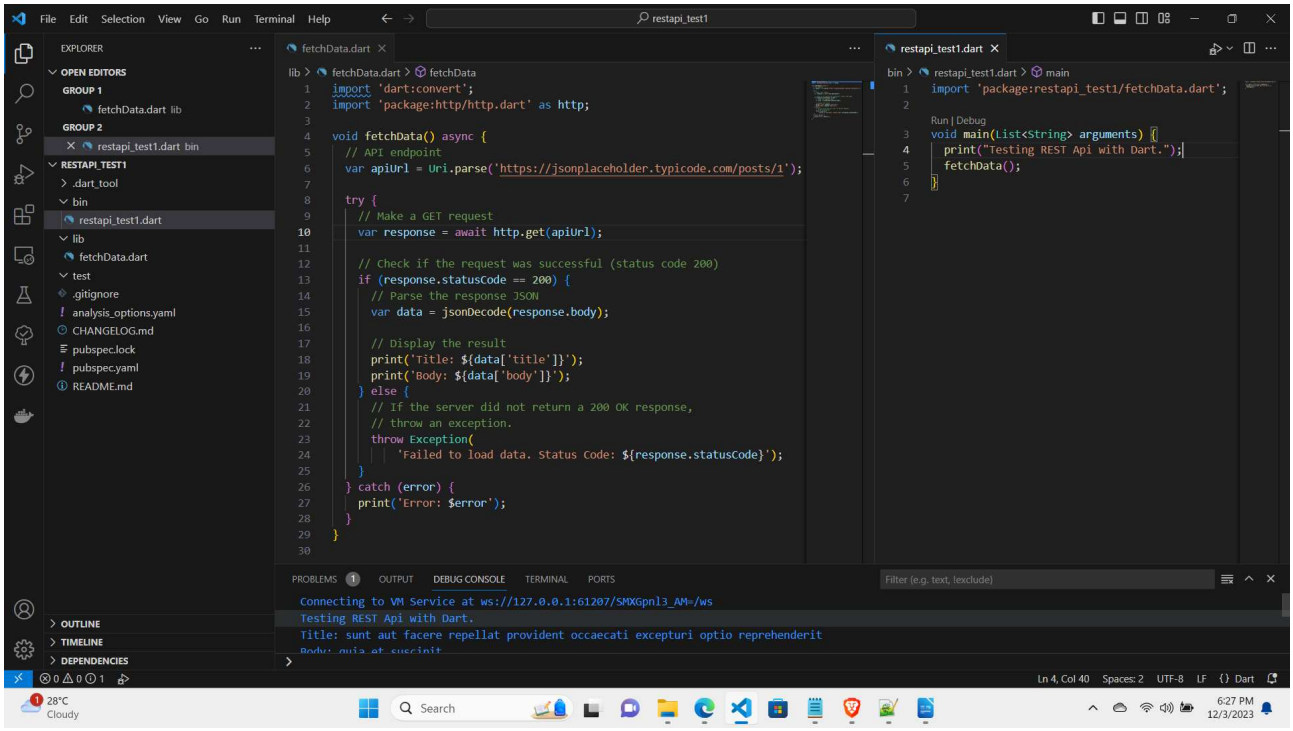
main() code:

```
import 'package:restapi_test1/fetchData.dart';

void main(List<String> arguments) {
  print("Testing REST API with Dart.");
  fetchData();
}
```

fetchData() function

```
import 'dart:convert';
import 'package:http/http.dart' as http;

void fetchData() async {
  // API endpoint
  var apiUrl = Uri.parse('https://jsonplaceholder.typicode.com/posts/1');

  try {
    // Make a GET request
    var response = await http.get(apiUrl);

    // Check if the request was successful (status code 200)
    if (response.statusCode == 200) {
      // Parse the response JSON
      var data = jsonDecode(response.body);

      // Display the result
      print('Title: ${data['title']}');
      print('Body: ${data['body']}');
    } else {
      // If the server did not return a 200 OK response,
      // throw an exception.
      throw Exception(
          'Failed to load data. Status Code: ${response.statusCode}');
    }
  } catch (error) {
    print('Error: $error');
  }
}
```

Observe the file structure.



Rest API to process a list of items

```
import 'dart:convert';
import 'package:http/http.dart' as http;

void fetchData() async {
  // Get all posts
  var apiUrl = Uri.parse('https://jsonplaceholder.typicode.com/posts');

  // Create an HTTP client
  final client = http.Client();
  try {
    // Send an HTTP GET request to the API endpoint
    final response = await client.get(apiUrl);

    // Check if the request was successful
    if (response.statusCode == 200) {
      // Parse the JSON response
      final data_list = jsonDecode(response.body) as List<dynamic>;

      //outputting the first element
      print("\nFirst element:");
      var first = data_list[0]; //use the first element
      print('Post ID: ${first['id']}');
      print('Title: ${first['title']}');
      print('Body: ${first['body']}\n');

      //outputting all elements
      print("\nAll element titles:");
      data_list.forEach((var post) {
        print('Title: ${post['title']}');
      });
    } else {
      print('Error retrieving data: ${response.statusCode}');
    }
  } catch (error) {
    print('Error: $error');
  } finally {
    print("This section will always run.");
  }
  client.close(); // Close the HTTP client
}
```

Reading a local file

We read a json file from the localdisk. Generally these resources are stored in “assests” folder. We cast the json text file into a Dart <List>. This is a common behavior to load constants from a file.

Main file

```
// bin/read_file.dart
import 'dart:convert';
import 'package:read_file/my_lib.dart' as my_lib;

void main(List<String> arguments) {
  print('Reading a file with async/await.');

  my_lib.readFile().then(
    (value) {
      print(
          "Showing all at once $value"); // Called when the Future completes successfully
      if (value != null) {
        print("Showing one at a time");
        final jsonData = jsonDecode(value) as List;
        jsonData.forEach((element) {
          print(element);
        });
      }
    },
    onError: (error) => print("Error: $error"), // Called when there's an error
  );
  print("This line executes immediately");
}
```

Library file

```
// lib/my_lib.dart
import 'dart:io';

Future<String?> readFile() async {
  try {
    final file = File('assets/cars.json');
    return await file.readAsString();
  } on IOException catch (e) {
    print('Error reading data file: $e');
  }
}
```

Testing Dart Applications

From: https://dart.dev/guides/testing

The Dart testing docs focus on three kinds of testing, out of the many kinds of testing that you may be familiar with: unit, component, and end-to-end (a form of integration testing). Testing terminology varies, but these are the terms and concepts that you are likely to encounter when using Dart technologies:

- Unit tests focus on verifying the smallest piece of testable software, such as a function, method, or class. Your test suites should have more unit tests than other kinds of tests.
- Component tests (called widget tests in Flutter) verify that a component (which usually consists of multiple classes) behaves as expected. A component test often requires the use of mock objects that can mimic user actions, events, perform layout, and instantiate child components.
- Integration and end-to-end tests verify the behavior of an entire app, or a large chunk of an app. An integration test generally runs on a simulated or real device or on a browser (for the web) and consists of two pieces: the app itself, and the test app that puts the app through its paces. An integration test often measures performance, so the test app generally runs on a different device or OS than the app being tested.

Testing Dart applications is a crucial aspect of the development process to ensure that your code behaves as expected and to catch potential issues early on. Dart provides a built-in testing library called `test` that makes it easy to write and run tests. Here's a summary of testing Dart applications:

1. Test Organization:
    - Tests are typically organized in separate Dart files with names ending in `_test.dart`.
    - Each test file contains one or more test cases, defined using the `test` function from the `test` library.

2. Writing Test Cases:
    - Test cases are created using the `test` function, which takes a description and a callback function.
    - The callback function contains the actual test logic, including assertions to verify expected behavior.

```
test('description of the test', () {
  // Test logic and assertions go here
});
```

3. Assertions:
    - Dart's testing library provides various assertion functions to check if values meet specific conditions.

- Common assertions include `expect`, `equals`, `isNotNull`, `isTrue`, `isFalse`, etc.

```
expect(actual, equals(expected));
```

4. Running Tests:

- Tests can be run using the `test` command in the terminal, specifying the path to the test file.
- Use the `dart test` command to run all tests in your Dart project.

```
dart test/my_test_file.dart
```

5. Test Coverage:

- Dart supports test coverage analysis to measure how much of your code is covered by tests.
- Use the `--coverage` option when running tests to generate coverage reports.

```
dart test –coverage=coverage
```

6. Setup and Teardown:

- Use the `setUp` and `tearDown` functions to set up and tear down resources before and after each test, respectively.

```
setUp(() {
  // Code to run before each test
});

tearDown(() {
  // Code to run after each test
});
```

7. Mocking and Test Doubles:

- Dart provides libraries like `mockito` for creating mocks and test doubles to isolate the code being tested.

```
// Example using mockito
final myMock = MockMyClass();
when(myMock.someMethod()).thenReturn('mocked value');
```

8. Continuous Integration (CI):

- Include tests in your CI/CD pipeline to ensure that code changes do not break existing functionality.
- Popular CI services like GitHub Actions and Travis CI support Dart testing.

Yaml code

```
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Set up Dart
        uses: dart-lang/setup-dart@v2
      - name: Run tests
        run: dart test
```

In summary, testing Dart applications involves organizing tests in separate files, writing test cases with assertions, running tests using the `test` library, and integrating testing into the development workflow and CI/CD pipelines. Testing helps catch bugs early, ensures code reliability, and facilitates maintainability as your Dart project grows.

References

- https://www.geeksforgeeks.org/introduction-to-dart-programming-language/
- https://www.javatpoint.com/dart-programming
- https://dart.dev/language
- https://jsonplaceholder.typicode.com
- Files and Folder Structure in Flutter & Dart: https://youtu.be/wR1cMW1hSzM
- https://medium.com/@ahsan-001/oop-in-flutter-in-depth-6cbf40640d69